Hand Gesture Detection Project Report

1. Introduction

This project involves building a hand gesture detection and control system using the YOLO object detection model, integrated with preprocessing steps using MediaPipe Hands. The final system detects hand gestures through the webcam and toggles various car-like control statuses (Engine, Indicators, Wiper, Headlights) based on gesture recognition.

2. Dataset

The dataset used for training the YOLO model was named 'hand-gesture-gizg2', downloaded from Roboflow. It contained approximately 8,000 images, labeled for six gesture classes corresponding to control commands: 0 (Engine Toggle), 1 (Indicator Right), 2 (Indicator Left), 3 (Wiper), 4 (Headlight Low Beam), and 5 (Headlight High Beam).

3. Model Training

We trained the YOLOv11 model on Google Colab using the prepared dataset. Training was performed for 50 epochs with an image size of 640, batch size 16, and checkpoints saved in Google Drive. Custom confidence thresholds were set for each class to improve model precision and reduce false detections.

4. Preprocessing & Filtering

During inference, MediaPipe Hands was used to extract the region containing the hand, while the rest of the frame was either blurred or made black. Additional processing included applying white light filters to suppress glare and improve detection accuracy.

5. Gesture Detection & Control Logic

Detections from YOLO were used to toggle various control states. A buffer stored recent detections, and actions were only triggered if the same gesture was detected in at least 60% of frames over a 3-second window. Controls implemented:

- Engine Toggle (with ON/OFF status)
- Indicator Right
- Indicator Left
- Wiper Toggle
- Low Beam Toggle
- High Beam Toggle

6. Real-time Display & Status Screen

A separate white window was created to display the current status of each control. Whenever a new command was executed, a message displayed at the bottom indicated the recent action performed.

7. Final Adjustments

Final improvements included:
- Suppressing YOLO log output to clean the terminal.
- Custom confidence thresholds for each class to improve accuracy (especially class 4 and 5).
- Addition of a white light filter to suppress bright glare regions.
- Option to capture a frame using the spacebar and trigger gesture detection manually.
- Stabilization of detection logic and removal of extra status prints in the frame.

8. Conclusion

The system was successfully implemented to detect and toggle car-like control statuses using hand gestures captured from a webcam in real-time. The integration of YOLO, MediaPipe Hands, and custom pre/post-processing led to robust performance suitable for prototype demonstration and further development.